

Statistical and Neural Machine Translation

A Condensed Tutorial for AI (Fundamental) Class Students

Dr. Ye Kyaw Thu (ရဲကျော်သူ)

Visiting Professor, Core AI Research Team, AI Research Group, NECTEC, Thailand
Lab Leader, Language Understanding Lab, AI Mario, Myanmar

Sept, 2026

Reference books: Koehn (2008), Koehn (2017)

Table of Contents

- 1 History and Motivation
- 2 Rule- and Example-Based MT
- 3 Statistical MT Foundations and IBM Models
- 4 Phrase, Hierarchical, and Operation-Sequence SMT
- 5 SMT Engineering: LM, Tuning, Decoding, Moses
- 6 Neural MT: RNNs, LSTM, and Attention
- 7 Transformer NMT and Practical Training
- 8 Pretrained and Multilingual MT
- 9 Evaluation, Low-Resource MT, and Modern Practice
- 10 Myanmar R&D, Practical Work, and Future Directions
- 11 References and Resources

What is Machine Translation?

Definition

Machine Translation (MT) is the use of software to automatically translate text or speech from one natural language (source) to another (target) without human intervention.

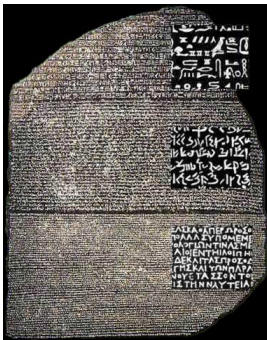
- One of the **oldest problems** in Artificial Intelligence
- Considered **AI-complete**: full understanding requires reasoning, world knowledge, and common sense
- Today, MT is a multi-billion-dollar industry (Google Translate, DeepL, ChatGPT, etc.)

Example

Myanmar example:

ကျွန်တော်သည် ကွန်ပျူတာသိပ္ပံဘာသာရပ်ကို လေ့လာနေသည်။
“I am studying Computer Science.”

The Origin of Parallel Corpora: The Rosetta Stone



The Rosetta Stone

Discovered in 1799, it contains the same decree in three scripts: Hieroglyphic, Demotic, and Greek. It is the most famous example of a **Trilingual Parallel Corpus**.



Jean-François Champollion

In 1822, Champollion deciphered the Hieroglyphs by **aligning** known Greek names with Egyptian symbols. This was the first "Statistical Alignment" in history.

The Rosetta Stone: A Parallel Text

- Discovered by French soldiers in **Rosetta (Egypt) in 1799** during Napoleon's Egyptian Campaign
- The stone contains a priestly decree (**196 BC**) written in **three scripts**:
 - 1 Ancient Egyptian hieroglyphics (formal script)
 - 2 Demotic script (everyday Egyptian)
 - 3 Ancient Greek (language of the rulers)
- It is essentially the world's most famous **parallel text (bilingual corpus)**
- Modern MT systems use the same idea: *learn from translated text pairs*

Key Insight

The Rosetta Stone allowed humans to *decipher* a forgotten language. Statistical MT allows computers to *learn* translation from millions of such parallel sentences.

Jean-François Champollion (1790–1832)

- French scholar and **linguist**
- On **14 September 1822**, Champollion announced his breakthrough in deciphering ancient Egyptian hieroglyphics
- He used the **Rosetta Stone** as the key: matching known Greek text against Egyptian scripts
- His method was essentially **alignment-based translation** — the same principle used in IBM Word Alignment Models (1993)!
- Published in *Lettre à M. Dacier* (1822)

*"It is a writing system
of signs, at the same
time figurative,
symbolic, and phonetic,
in the same text,
the same sentence,
the same word."*

— Champollion, 1822

Connection to MT

Champollion's insight: a **known translation** can reveal the structure of an unknown language. This is the *statistical alignment* idea, 170 years before IBM models.

The Warren Weaver Memorandum (1947)

- **Warren Weaver** (Rockefeller Foundation) wrote a famous memo in 1947 proposing the use of computers for translation
- Inspired by success of **code-breaking** in World War II (e.g., Enigma)
- Key ideas in the memo:
 - 1 Translation as a **cryptographic** problem
 - 2 Use of **statistical analysis** of language
 - 3 Universal underlying **meaning** across languages
- The memo said: *"When I look at an article in Russian, I say: This is really written in English, but it has been coded in some strange symbols."*

Weaver's Vision (1947)

- Computers can decode foreign text
- Statistical patterns reveal meaning
- The **noisy channel model** idea!

This vision became reality 45 years later with **IBM models (1993)**.

Georgetown-IBM Experiment (1954)

- **January 7, 1954:** First public demonstration of MT at **IBM headquarters, New York**
- Developed by Georgetown University and IBM
- The system translated **60 Russian sentences into English**
- Used only:
 - **6 grammar rules**
 - **250 vocabulary items**
- Covered topics: politics, law, mathematics, chemistry, metallurgy

Optimistic Predictions (1954)

Researchers predicted MT would be “**solved in 3–5 years**”. This optimism led to massive Cold War funding from both the USA and USSR for MT research.

Reality Check

The problem proved far harder than expected. True high-quality MT was not achieved until the **2010s** with neural methods.

United States (1950s–1960s):

- Funded by **NSF, DARPA, CIA, DoD**
- Goal: translate Russian scientific literature quickly
- Many universities built RBMT systems
- Key projects: SYSTRAN, Mark II

Soviet Union:

- Parallel development in USSR
- Goal: translate US military and scientific documents
- Work at Moscow State University and other institutes

ALPAC Report (1966)

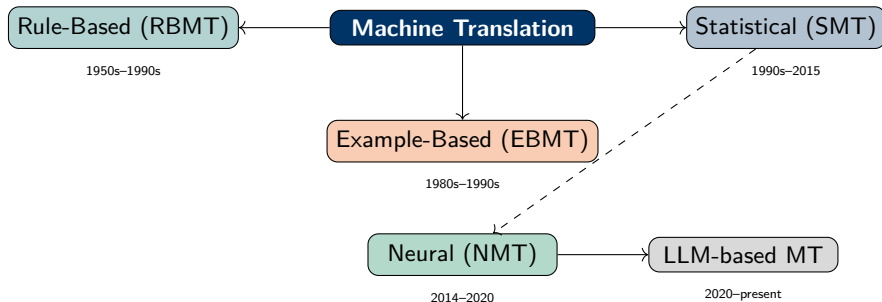
The **Automatic Language Processing Advisory Committee** (ALPAC) report concluded that:

- MT was **twice as expensive** as human translation
- **Twice as slow**
- Of **lower quality**

⇒ US government **stopped funding MT** research for almost a decade.

⇒ Known as the “**MT winter**”

MT Approaches: An Overview



MT History Timeline

1822	Charnpallion deciphers Egyptian hieroglyphics using Rosetta Stone
1947	Warren Weaver's memorandum proposes statistical MT
1954	Georgetown-IBM experiment: first public MT demo
1966	ALPAC report halts US MT funding
1970s	Rule-based systems (SYSTRAN, LOGOS) for practical use
1988	IBM proposes statistical/noisy channel approach to MT
1993	Brown et al. publish IBM Models 1–5 (<i>Computational Linguistics</i>)
2002	Papineni et al. introduce BLEU evaluation metric
2003	Koehn et al. propose Phrase-Based SMT
2006	Moses open-source toolkit released (JHU summer workshop)
2007	Chiang proposes Hierarchical Phrase-Based MT (Hiero)
2014	Cho et al., Sutskever et al. propose encoder-decoder NMT
2015	Bahdanau et al. introduce attention mechanism for NMT
2017	Vaswani et al. " Attention is All You Need " (Transformer)
2020	mBART, mT5 and large pretrained multilingual models
2022	Meta AI releases NLLB-200 (200 languages)

Why is MT Hard? Challenges

Linguistic Challenges:

- **Ambiguity:** “I saw the man with the telescope”
- **Idioms:** “kick the bucket” $\neq$ literal
- **Reordering:** SOV vs. SVO languages
- **Morphology:** agglutinative languages (Turkish, Finnish, Myanmar)
- **Pronoun dropping:** null-subject languages

Computational Challenges:

- **Search problem:** exponential translation space
- **Data sparsity:** low-resource language pairs
- **Domain adaptation:** general $\neq$ medical/legal
- **Evaluation:** no single “correct” translation
- **Context:** coreference across sentences

Myanmar Example —Ambiguity

ခေါင်းထဲမှာ ဘာရှိလဲ။

English: “What is inside the **head**?” or “What is inside the **coffin**?”

(Context disambiguates!)

Rule-Based MT (RBMT) —Overview

Definition

Rule-Based MT uses hand-crafted **linguistic rules** (grammar, morphology, syntax) and **bilingual dictionaries** to translate text.

Three main paradigms:

- 1 **Direct Transfer** —word-for-word replacement using dictionaries
- 2 **Transfer-Based** —parse source, transfer structure, generate target
- 3 **Interlingual** —convert source to a language-neutral representation

Well-known RBMT systems:

- **SYSTRAN** (1968, used by the EU and US military)
- **LOGOS** (1970s, commercial)
- **METAL** (Siemens, 1980s)
- **OpenLogos** (open-source version of LOGOS)

Direct Transfer RBMT

Approach: Translate word by word using a bilingual dictionary, with minimal structural analysis.

Steps:

- 1 Tokenize the source sentence
- 2 Look up each word in the dictionary
- 3 Apply simple morphological rules
- 4 Output the translated words in source order

Example

Source (Myanmar): ကတ်ကို ဖြတ်ပါ

Word lookup:

- ကတ် → card
- ကို → (object marker, skip)
- ဖြတ် → cut
- ပါ → (polite, skip)

Output: “card cut” (incorrect word order!)

Limitation

Direct transfer ignores syntax, so it fails badly for language pairs with different word order (e.g., Myanmar SOV vs. English SVO).

Transfer-Based and Interlingual RBMT

Transfer-Based RBMT:

- ➊ **Analysis:** Parse source sentence (morphology + syntax)
- ➋ **Transfer:** Map source parse tree to target parse tree using bilingual transfer rules
- ➌ **Generation:** Generate target sentence from target parse tree

Better than direct, but transfer rules are **language-pair specific**.

RBMT Advantages

- Predictable and controllable output
- Good for well-defined domains
- No training data required

Interlingual RBMT:

- ➊ **Analysis:** Convert source to universal *interlingua* (language-neutral representation)
- ➋ **Generation:** Generate any target language from interlingua

Very powerful in theory; **but constructing a universal interlingua is extremely difficult** in practice.

RBMT Disadvantages

- Requires expert linguists (expensive)
- Hard to maintain and scale
- Coverage is limited by rules
- Poor handling of ambiguity

Example-Based MT (EBMT)

Concept

EBMT was proposed by Makoto Nagao (1984). It translates by **analogy**: find similar source sentences in a *translation memory*, adapt their translations to the new input.

EBMT Steps:

- ➊ **Match**: Find the most similar source sentence(s) in the example base
- ➋ **Adapt**: Modify the matched translation to fit the new source sentence
- ➌ **Combine**: Merge adapted fragments into the final translation

Analogy Example

Known: “私は東京に行く” → “I go to Tokyo”

Input: “私は大阪に行く”

Match: Same except 東京 → 大阪

Output: “I go to Osaka”

Advantages

- Human-quality translations for seen cases
- Reuses validated translations
- Easy to interpret

Limitations

- Fails on unseen sentences
- Brittle combination step
- Hard to scale

- EBMT ideas live on in modern **translation memory (TM)** systems used in professional CAT tools (e.g., SDL Trados, memoQ)
- The concept of **phrase reuse** is directly incorporated in Phrase-Based SMT
- Modern NMT systems sometimes augment with TM for high-precision domains
- **Retrieval-Augmented MT** combines neural MT with example retrieval

Key Idea Carried Forward

EBMT showed that “**learning from examples**” is a valid strategy. Statistical MT formalizes this idea with probability theory.

Why Statistical Machine Translation?

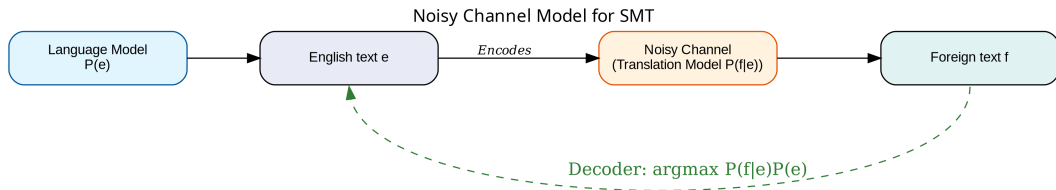
- **Data is cheap; rules are expensive**
- Billions of parallel sentences exist online (EU proceedings, UN documents, Wikipedia, news archives)
- Statistical models **learn translation patterns automatically**
- Can handle **ambiguity probabilistically**
- Easy to **extend to new language pairs** (just add data)

The Fundamental SMT Insight (IBM, 1988–1993)

$$\begin{aligned}\hat{e} &= \arg \max_e P(e \mid f) \\ &= \arg \max_e \underbrace{P(f \mid e)}_{\text{Translation Model}} \cdot \underbrace{P(e)}_{\text{Language Model}}\end{aligned}$$

Noisy Channel Model: imagine e (English) was “garbled” (encoded) into f (foreign). We want to recover the most likely e .

The Noisy Channel Model —Explained Simply



Three components of SMT:

- 1 **Translation Model** $P(f | e)$: How likely is the foreign sentence given the English? Learned from parallel corpora.
- 2 **Language Model** $P(e)$: How likely is the English sentence? Learned from monolingual English text.
- 3 **Decoder**: Search algorithm that finds the best $\hat{e}$ that maximizes the product.

The Noisy Channel Model —Explained Simply

Three components of SMT:

- 1 **Translation Model** $P(f|e)$: probability of source given target. Learned from parallel corpora.
- 2 **Language Model** $P(e)$: how fluent is the English output? Learned from monolingual text.
- 3 **Decoder**: finds $\hat{e} = \arg \max_e P(f|e) P(e)$.

Why this works

$P(f|e)$ ensures the translation is **faithful** to the source.

$P(e)$ ensures the translation is **fluent** English.

BLEU: Automatic MT Evaluation

BLEU Score (Papineni et al., 2002)

Bilingual Evaluation Understudy —the standard automatic metric for MT quality.

Intuition: Compare MT output against human reference translations by counting **n-gram overlaps**.

$$\text{BLEU} = \underbrace{\text{BP}}_{\text{Brevity Penalty}} \times \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

- p_n = modified precision for n -grams (1, 2, 3, 4)
- $w_n = 1/N$ (uniform weights, typically $N = 4$)
- $\text{BP} = 1$ if output length $\geq$ reference; $e^{1-r/c}$ otherwise
- Score: 0 (worst) to 100 (best); human level $\approx 50+$

Caution

BLEU is not perfect! It does not capture meaning, fluency, or adequacy directly. Always combine with human evaluation for important decisions.

BLEU: Bilingual Evaluation Understudy

- **Method:** Counts overlapping n-grams (1-gram, 2-gram, etc.)

Reference (Human):

ကျွန်တော် ထမင်း စားသည်

MT Output (Machine):

ကျွန်တော် စားသည် ထမင်း

N-gram	Matches	Precision
1-gram	ကျွန်တော်, စားသည်, ထမင်း	$3/3 = 1.0$
2-gram	(none match)	$0/2 = 0.0$

Takeaway

Even if all words are correct (1-gram=1.0), the score drops if the **order** is wrong (2-gram=0.0). This is why BLEU captures "fluency."

Word Alignment —The Core Concept

What is Word Alignment?

A word alignment a maps each source word f_j to a target word $e_{a(j)}$ (or to a special **NULL** word if f_j is untranslated).

Example alignment:

English:	Mary	did not	slap	the	witch
	↓	↓	↓	↓	↓
Spanish:	Maria	no	daba	una	bruja

Myanmar Example

English: I love Myanmar

Myanmar: ကျွန်တော် မြန်မာကို ချစ်သည်

Alignment: $a(1) = 1, a(2) = 3, a(3) = 2$

Note the **reordering**: verb comes last in Myanmar (SOV)!

IBM Model 1 —The Simplest Model

Key assumption: alignment probability is **uniform** —any source word is equally likely to align to any target word.

IBM Model 1 Formula

$$P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \frac{\epsilon}{(I+1)^m} \prod_{j=1}^m t(f_j \mid e_{a(j)}) \quad (2)$$

- $t(f \mid e)$ = **lexical translation probability** (how likely is f a translation of e ?)
- ϵ = a small constant
- $(I+1)^m$ = normalization over all possible alignments
- I = length of English sentence (target)
- m = length of foreign sentence (source)

IBM Model 1 —The Simplest Model

Summing over all alignments:

$$P(\mathbf{f} \mid \mathbf{e}) = \sum_{\mathbf{a}} P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=0}^l t(f_j \mid e_i) \quad (3)$$

EM Algorithm —Small Example

Suppose we have two parallel sentences:

① “the house” $\leftrightarrow$ “အိမ်”

② “the dog” $\leftrightarrow$ “ခွေး”

Initialization: All $t(f|e) = 0.5$ (uniform)

Iteration 1 E-step: Compute soft counts:

Foreign	English	Soft count
အိမ်	the	0.5
အိမ်	house	0.5
ခွေး	the	0.5
ခွေး	dog	0.5

Iteration 1 M-step: Normalize $\Rightarrow$ still uniform

After many iterations: $t(\text{အိမ်} | \text{house}) \approx 1$, $t(\text{ခွေး} | \text{dog}) \approx 1$
 $\Rightarrow$ the article *the* is split between both foreign words equally.

IBM Model 3 —Adding Fertility

New concept: Fertility ϕ_i = number of foreign words that English word e_i generates (can be 0, 1, 2, ...).

IBM Model 3 Formula

$$P(\mathbf{f}, \mathbf{a} \mid \mathbf{e}) = \prod_{i=1}^I \underbrace{n(\phi_i \mid e_i)}_{\text{fertility}} \cdot \phi_i! \cdot \prod_{j:a(j)=i} t(f_j \mid e_i) \cdot \prod_{j=1}^m d(j \mid a(j), m, I) \quad (4)$$

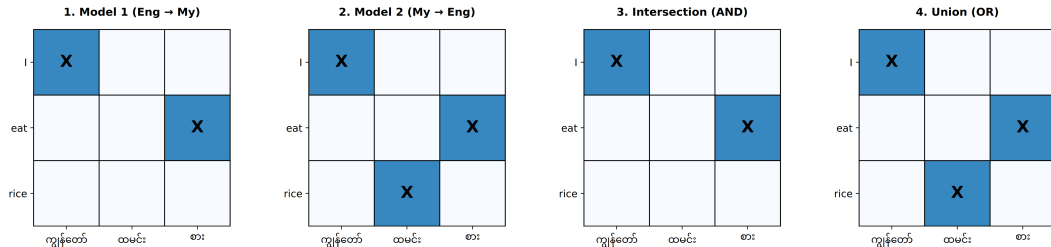
Three new parameters:

- $n(\phi \mid e)$: probability that word e has fertility ϕ
- $d(j \mid i, m, I)$: **distortion** probability (where source word lands)
- p_0, p_1 : probabilities for inserting NULL-generated words

Example

English “the” may have fertility 0 when translating to Myanmar (no article in Myanmar language).
English “not” may generate two words in some languages (double negation). e.g., မှားမှား

SMT Word Alignment Symmetrization



- In SMT, directional models (IBM Models) are “many-to-one.” English → Myanmar might fail to find a word because of a complex phrase structure.
- * The **Intersection** is very “safe” (high precision) but usually leaves many words unaligned.
- * The **Union** is very “greedy” (high recall) and ensures more words are covered, which is essential for building a complete Phrase Table.

Grow-Diag-Final

	1. Model 1 (E→M)				2. Model 2 (M→E)				3. Union (Too Noisy)				4. GDF (Perfect)			
I	X				X		X		X				X			
ate			X					X		X	X			X	X	
rice		X			X				X					X		
	ကျွန်တော်	ထမင်း	စား	ခဲတယ်	ကျွန်တော်	ထမင်း	စား	ခဲတယ်	ကျွန်တော်	ထမင်း	စား	ခဲတယ်	ကျွန်တော်	ထမင်း	စား	ခဲတယ်

- ****Intersection:**** Misses the verb.
- ****Union:**** The Union is “too greedy”—it accepts every mistake from both models.
- ****Grow-Diag-Final (GDF):**** It grows to catch the verb but ignores the noise.

Phrase Extraction Algorithm

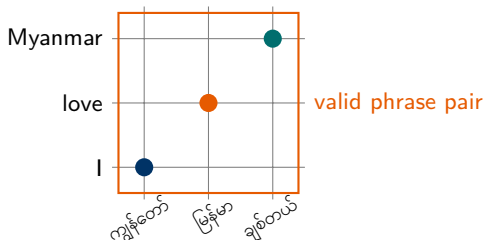
Step 1: Obtain word alignments (GIZA++ bidirectional)

Step 2: Extract all **consistent phrase pairs** from alignments

Consistency Condition

A phrase pair $(\bar{f}, \bar{e})$ is **consistent** with alignment a if:

- 1 Every foreign word in $\bar{f}$ that is aligned maps to a word in $\bar{e}$
- 2 Every target word in $\bar{e}$ that is aligned maps to a word in $\bar{f}$
- 3 At least one word in $\bar{f}$ is aligned



The Log-Linear Model for Phrase-Based SMT

Log-Linear Combination (Och & Ney, 2002)

$$P(\mathbf{e} \mid \mathbf{f}) \propto \exp \left(\sum_{i=1}^n \lambda_i \cdot h_i(\mathbf{e}, \mathbf{f}) \right) \quad (5)$$

The best translation is:

$$\hat{\mathbf{e}} = \arg \max_{\mathbf{e}} \sum_{i=1}^n \lambda_i \cdot h_i(\mathbf{e}, \mathbf{f}) \quad (6)$$

Standard feature functions h_i :

- | | |
|---|--|
| ❶ h_1 : Phrase translation probability $\log \phi(\bar{f} \mid \bar{e})$ | ❺ h_5 : Language model $\log P_{LM}(\mathbf{e})$ |
| ❷ h_2 : Inverse phrase probability $\log \phi(\bar{e} \mid \bar{f})$ | ❻ h_6 : Word penalty |
| ❸ h_3 : Lexical weighting $\log \text{lex}(\bar{f} \mid \bar{e})$ | ❼ h_7 : Phrase penalty |
| ❹ h_4 : Inverse lexical weighting $\log \text{lex}(\bar{e} \mid \bar{f})$ | ❽ h_8 : Reordering model |

- Source and target languages often have different word orders
- **Monotonic** translation: phrases in same order as source
- **Reordering**: some phrases are swapped

Distance-Based Reordering (Simple)

Penalize **large jumps** in phrase alignment:

$$d(\mathbf{e}, \mathbf{f}) = \exp \left(-\lambda \sum_{\text{phrase}} |e_{\text{start}} - f_{\text{prev_end}}| \right) \quad (7)$$

Practical Example: Distortion Penalty (Reordering)

- **Concept:** How far does the "cursor" jump to find the next word?
- **Formula:** $d_i = |\text{start}_i - \text{end}_{i-1} - 1|$

Python Implementation: English (SVO) → Myanmar (SOV)

```
def calculate_distortion(prev_end, curr_start):  
    # |current_start - prev_end - 1|  
    return abs(curr_start - prev_end - 1)  
  
# English: I(0) eat(1) rice(2) -> Myanmar: I(0) rice(1) eat(2)  
# 1. Translate "I" (Source Pos 0)  
d1 = calculate_distortion(-1, 0) # Score: 0  
  
# 2. Translate "rice" (Source Pos 2)  
d2 = calculate_distortion(0, 2) # Score: 1 (Jump!)  
  
# 3. Translate "eat" (Source Pos 1)  
d3 = calculate_distortion(2, 1) # Score: 2 (Jump back!)  
  
total_penalty = d1 + d2 + d3 # Total: 3
```

Intuition for Students

In SMT, we want to **minimize** this penalty. A very high score suggests the translation might be ungrammatical or the alignment is too chaotic.

Phrase-Based SMT: Summary & Limitations

Strengths

- Handles multi-word expressions
- Local context through phrases
- Strong empirical performance (2003–2014)
- Interpretable (phrase table visible)
- Moses: easy to train and use

Limitations

- Limited **long-range reordering**
- Phrase table can be huge
- Independence between phrases
- Poor morphology handling
- No global syntactic structure

⇒ These limitations motivated **Hierarchical Phrase-Based SMT** and eventually **Syntax-Based SMT**.

Beyond Hiero, **linguistically-informed** syntax models:

String-to-Tree (Galley et al., 2006):

- Source: flat string
- Target: parsed tree structure
- Uses target-language parser

Tree-to-String (Liu et al., 2006):

- Source: parsed tree
- Target: flat string
- Uses source-language parser

Tree-to-Tree (Yamada & Knight, 2001):

- Both sides parsed
- Highest quality but requires two parsers
- Very expensive

Moses Support

Moses includes both phrase-based and tree-based (Hiero) decoding. Select with `-search-algorithm 1` for CKY.

Hiero vs. Phrase-Based: Key Differences

Aspect	Phrase-Based	Hiero
Translation units	Flat phrases	Hierarchical rules
Reordering	Local (distortion limit)	Long-range (via grammar)
Grammar	None	SCFG
Decoding	Stack-based	CKY chart parsing
Syntax	No	Implicit
Speed	Faster	Slower
Rule count	Millions of phrases	Even more rules
Performance	Good	Better for distant pairs

When to Use Hiero?

Hiero tends to outperform phrase-based MT for language pairs with **very different word orders**, such as Chinese-English, Japanese-English, or **Myanmar-English** (SOV↔SVO).

OSM Probability

$$P(\mathbf{o}_1^n) = \prod_{k=1}^n P(o_k \mid o_{k-N+1}, \dots, o_{k-1}) \quad (8)$$

$\mathbf{o}_1^n$ is the sequence of operations; modeled as an **N-gram language model over operations**.

Key advantages over phrase-based:

- Reordering and translation are **jointly modeled**
- No need for separate reordering model
- Captures **long-distance** dependencies through operation sequences
- Can be added as a **feature** in Moses alongside phrase-based

Research Result (Ye Kyaw Thu et al., 2021)

Compared Phrase-based, Hierarchical, and OSM for Burmese–Pa'O language pair. All three models were combined (system combination) for best results.

Reference: “SMT System Combinations on Phrase-based, Hierarchical Phrase-based and OSM for Burmese and Pa'O Language Pair”, JIIST 2021.

OSM for Myanmar–Pa'O Translation:

- Pa'O is a Tibeto-Burman language spoken in Shan State, Myanmar
- SOV word order (similar to Myanmar)
- Small parallel corpus (low-resource setting)
- OSM captures operation patterns unique to this pair

System combination (Moses):

- Phrase-based + Hierarchical + OSM combined using **MERT**
- Combined system outperforms individual models

OSM vs. Phrase-based (BLEU)

System	bm-po	po-bm
Phrase-based	33.04	41.20
Hierarchical	33.23	41.70
OSM	35.06	43.45
Combined 3	38.45	47.57

Role of the Language Model

The Language Model (LM) scores how **fluent** and **grammatical** the target language output is, independently of the source sentence.

$$P(\mathbf{e}) = P(e_1, e_2, \dots, e_l) \quad (9)$$

A good LM assigns high probability to grammatical English and low probability to word salad.

- Trained on **large monolingual** target-language corpora
- For English: billions of words (Common Crawl, Wikipedia, news)
- For Myanmar: smaller corpora, but growing
- The LM is the “spell-checker” of MT —it ensures fluent output

N-gram Calculation: Step-by-Step

Training Corpus:

- ❶ ကျွန်တော် ထမင်း စားသည်။
- ❷ ကျွန်တော် ရေ သောက်သည်။
- ❸ မောင်မောင် ထမင်း စားသည်။

Bigram Probability: $P(\text{ထမင်း}|\text{ကျွန်တော်})$

Formula: $\frac{\text{Count}(\text{ကျွန်တော်}, \text{ထမင်း})}{\text{Count}(\text{ကျွန်တော်})}$

Calculation: $\frac{1}{2} = 0.5$

Trigram Probability: $P(\text{စားသည်}|\text{ကျွန်တော်}, \text{ထမင်း})$

Formula: $\frac{\text{Count}(\text{ကျွန်တော်}, \text{ထမင်း}, \text{စားသည်})}{\text{Count}(\text{ကျွန်တော်}, \text{ထမင်း})}$

Calculation: $\frac{1}{1} = 1.0$

The Sparsity Problem & Back-off

- **Problem:** What if we see “ကျွန်တော် ရေ စားသည်”?
- Our data has 0 counts for this trigram → Probability becomes **0**.
- A zero probability in any part of the sentence makes the **whole sentence** probability 0.

Simple Back-off Logic

If $\text{Count}(w_1, w_2, w_3) == 0$:

- 1 "Back-off" to Bigram: $P(w_3|w_2)$
- 2 Apply a penalty weight (α): $\alpha \times P(w_3|w_2)$

Intuition: If we don't know the full phrase, we guess based on the last two words.

Smoothing: Handling Unseen N-grams

Problem: Many n-grams never appear in training data, so $P(e_i | e_{i-1}, \dots) = 0$, making the whole sentence probability 0. **Solution: Smoothing** —redistribute probability mass to unseen events.

Major Smoothing Techniques

Laplace (add-one) Add 1 to all counts:

$$P_{\text{Lap}}(e_i | e_{i-1}) = \frac{c(e_{i-1}, e_i) + 1}{c(e_{i-1}) + V}$$

Jelinek-Mercer Linearly interpolate different n-gram orders:

$$P_{\text{JM}} = \lambda_n P_n + (1 - \lambda_n) P_{n-1}$$

Kneser-Ney (KN) Use **continuation probability** for backoff:

Best-performing in practice; standard for SMT

Modified KN Chen & Goodman (1998) —even better

Kneser-Ney Smoothing: Beyond Simple Counting

- Standard smoothing only looks at **Frequency**.
- Kneser-Ney looks at **Continuation Probability** (Versatility).

High Frequency

“သည်” (Sentence closer)

Appears often, but only in ONE context (at the end).

High Versatility

“စားသည်” (to eat)

Appears with “ထမင်း”, “မုန့်”, “အသီး”. More likely to follow a new word.

The Logic: If we encounter a new word, it is more likely to be followed by a word that appears in **many different contexts** rather than just a word that is common in one spot.

Kneser-Ney Smoothing —Intuition

Example 1: San Francisco

“Francisco” appears frequently in corpora, but almost always follows “San”. Its **continuation probability** is low because it is not a “versatile” word.

Example 2: Myanmar Context (မြန်မာဘာသာဖြင့် ဥပမာ)

“ဂိတ်” (Station) ဟူသောစကားလုံးသည် စာသားထဲတွင် အကြိမ်ရေများစွာပါသော်လည်း များသောအားဖြင့် “မီးရထား” သို့မဟုတ် “ကား” နောက်တွင်သာ တွဲလျက်ပါလေ့ရှိသည်။

- **High Freq, Low Continuation:** “ဂိတ်” သည် Context အနည်းငယ်တွင်သာ ပေါ်တတ်သဖြင့် Backoff ပေးရာတွင် ဦးစားမပေးသင့်ပါ။
- **Low Freq, High Continuation:** “နား” (Ear/Near) ကဲ့သို့ စကားလုံးမျိုးသည် Context ပေါင်းစုံတွင် ပါဝင်နိုင်သဖြင့် Continuation probability ပိုမြင့်ပါသည်။

Key Takeaway (မှတ်ချက်)

စကားလုံးတစ်လုံး၏ ပေါ်ထွက်နှုန်း (Raw Count) တစ်ခုတည်းကို မကြည့်ဘဲ၊ ၎င်းသည် ရှေ့ဆက်စကားလုံး ဘယ်နှစ်မျိုး နှင့် တွဲဖက်နိုင်သနည်း (Diversity of contexts) ကို ကြည့်ခြင်းဖြစ်သည်။

Why Optimize Feature Weights?

- The log-linear model combines many features: phrase probabilities, LM, reordering, word penalty ...
- Each feature has a **weight** λ_i that determines its importance
- **Bad weights** $\Rightarrow$ bad translations (even with good features)
- We want to **automatically find** the best weights

Optimization Goal

Find weights $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_n)$ that maximize translation quality on a **development set**:

$$\hat{\boldsymbol{\lambda}} = \arg \max_{\boldsymbol{\lambda}} \text{BLEU}(\boldsymbol{\lambda}) \quad (10)$$

Problem: BLEU is **not differentiable** — can't use gradient descent directly.

PRO and MIRA: Modern Optimization

PRO (Hopkins & May, 2011)

Pairwise Ranking Optimization:

- Sample pairs of translations from k -best list
- Label them (better/worse) based on BLEU
- Train a classifier (logistic regression) to rank correctly
- Works with **hundreds of features**

MIRA (Chiang et al., 2008)

Margin Infused Relaxed Algorithm:

- Online perceptron-style algorithm
- Update weights when a better translation is ranked lower
- Very fast convergence
- Good for **sparse feature sets** (millions of features)

Running MERT in Moses

```
perl mert-moses.pl devset.src devset.ref \  
  moses /path/to/moses.ini \  
  --decoder-flags "-threads_4" \  
  --mertdir /path/to/mert
```

Training Pipeline: Complete SMT Workflow

- 1 **Data preparation:** tokenize, lowercase, clean parallel corpus
- 2 **Word alignment:** run GIZA++ in both directions ($e \rightarrow f$, $f \rightarrow e$)
- 3 **Symmetrize alignments** (grow-diag-final-and)
- 4 **Extract phrases** and compute phrase probabilities
- 5 **Train language model** (KenLM 5-gram) on target monolingual data
- 6 **Build Moses configuration** file (moses.ini)
- 7 **Optimize weights** with MERT on development set
- 8 **Evaluate** on test set (BLEU, TER, METEOR)

Moses train-model.perl

```
perl train-model.perl -root-dir /path/to/model \  
-corpus /path/to/parallel \  
-f my -e en \  
-alignment grow-diag-final-and \  
-reordering msd-bidirectional-fe \  
-lm 0:5:/path/to/lm.blm:8 \  
-external-bin-dir /tools/bin
```

Beam Search in SMT Decoding

Beam search keeps only the top- b hypotheses at each step:

- **Beam size** b : trade-off between quality and speed
- $b = 1$: greedy search (fast but often bad)
- $b = 200$: standard Moses default
- Larger b = better quality but slower

Hypothesis score:

$$\text{score}(h) = \sum_i \lambda_i h_i + \text{future_cost}(h) \quad (11)$$

Future cost: **heuristic estimate** of completing the translation.

Pruning Strategies

- **Beam pruning**: keep top- b by score
- **Histogram pruning**: hard limit per stack
- **Threshold pruning**: drop hypotheses within α of best
- **Recombination**: merge identical coverage+end states

Hypothesis Recombination

Key optimization: Two hypotheses can be **merged** if they are **equivalent** for future decisions:

Recombination Condition

Two hypotheses h_1 and h_2 are equivalent (and the weaker is discarded) if they have:

- 1 The **same set of covered source words**
- 2 The **same last $n - 1$ target words** (same LM state)

Keep only the higher-scoring one.

This reduces the search space dramatically without losing the optimal solution.

Decoding Output

Moses can output:

- **1-best:** highest scoring translation
- **N-best list:** top- N translations with scores (useful for reranking)
- **Word lattice:** compact representation of all translations

Reordering Constraints in Decoding

Problem: allowing arbitrary reordering is computationally expensive.

Distortion Limit (Moses)

Moses limits reordering to within d positions (default: $d = 6$):

$$|e_{\text{start}} - f_{\text{prev_end}}| \leq d \quad (12)$$

- $d = 0$: fully monotonic (fastest, worst quality)
- $d = 6$: standard default (good balance)
- $d = \infty$: unlimited reordering (slowest)

For Myanmar–English

Myanmar (SOV) to English (SVO) requires significant reordering. Use larger distortion limit ($d \geq 10$) or switch to Hiero for long-distance reordering.

Moses: External Components

Word Alignment:

- GIZA++ (IBM Models)
- fast_align (faster, similar quality)
- Berkeley Aligner (discriminative)

Language Model:

- KenLM (default, fast)
- SRILM (more options)
- RandLM (randomized)

Evaluation:

- mteval-v13a.pl (BLEU)
- TER-plus
- Multi-BLEU

Other Tools:

- Tokenizers (many languages)
- sentence aligner (Hunalign)
- Corpus cleaning scripts

Moses: Complete Training Example

Step 1: Prepare data

```
# Tokenize
tokenizer.perl -l en < train.en \
  > train.tok.en
# Lowercase
lowercase.perl < train.tok.en \
  > train.lc.en
# Clean (max 80 words)
clean-corpus-n.perl train.lc my en \
  train.clean 1 80
```

Step 2: Train LM

```
lmplz -o 5 < train.lc.en \
  > lm.arpa
build_binary lm.arpa lm.blm
```

Step 3: Train model

```
perl train-model.perl \
  -root-dir model \
  -corpus train.clean \
  -f my -e en \
  -alignment \
    grow-diag-final-and \
  -reordering \
    msd-bidirectional-fe \
  -lm 0:5:lm.blm:8 \
  -mgiza \
  -external-bin-dir tools
```

What We Learned about SMT

- ① **Noisy channel model**: decompose into Translation Model + Language Model
- ② **IBM Models 1–4**: word alignment via EM algorithm
- ③ **Phrase-based SMT**: translate phrases, not words; use log-linear features
- ④ **Hierarchical SMT**: hierarchical rules for long-range reordering
- ⑤ **OSM**: joint model of translation + reordering operations
- ⑥ **Language Model**: n-gram with Kneser-Ney smoothing
- ⑦ **MERT**: optimize feature weights for BLEU
- ⑧ **Moses**: the standard open-source SMT toolkit

Now we shift to Neural Machine Translation...

NMT History: Early Attempts

- **1990s:** Early neural MT attempts
 - Waibel et al. (1991): connectionist models for speech translation
 - Forcada and Neco (1997): RNN-based MT (strikingly similar to modern approaches!)
 - But: insufficient data, insufficient compute $\Rightarrow$ abandoned
- **2007:** Schwenk proposes **neural language models** for SMT
 - Feed-forward NN-LM improves SMT quality significantly
 - Large improvement in evaluation campaigns
- **2012–2013:** Neural features integrated into SMT components (Devlin et al., Schwenk, others)
- **2013:** Kalchbrenner & Blunsom — CNN-based encoder-decoder
- **2014:** **Cho et al. and Sutskever et al.** — full neural MT breakthrough
- **2015:** **Bahdanau et al.** — attention mechanism resolves long-sentence problem
- **2016:** NMT wins WMT in nearly all language pairs
- **2017:** **Transformer** makes NMT much faster and better

Paper

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014).

“Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation.”

EMNLP 2014.

- Proposed the **RNN Encoder-Decoder** (Seq2Seq) architecture
- Variable-length input → **fixed-length context vector** → variable-length output
- Also introduced the **GRU (Gated Recurrent Unit)**
- Initially used as an **additional feature** for SMT; later used standalone

Simultaneously

Sutskever et al. (2014) at Google proposed a similar “Sequence to Sequence” model using **LSTM** cells — another landmark paper published at NIPS 2014.

The Bottleneck Problem

Critical Weakness

The entire source sentence meaning must be compressed into a **single fixed-size context vector c** .

For long sentences, this **bottleneck** causes information loss and translation quality degrades significantly.

Evidence (Cho et al., 2014):

- Translation quality is good for short sentences (up to 20 words)
- Quality drops sharply for sentences > 30 words
- The fixed-size vector simply cannot hold all information

Solution

Bahdanau et al. (2015) introduced the **attention mechanism** to solve this bottleneck: instead of one vector, use **all encoder hidden states** and **selectively focus** on relevant parts.

Training NMT: Maximum Likelihood

Training Objective

Minimize **cross-entropy** (equivalent to maximizing log-likelihood):

$$\mathcal{L} = - \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}} \sum_{t=1}^T \log P(y_t \mid y_1, \dots, y_{t-1}, \mathbf{x}) \quad (13)$$

Train using **teacher forcing**: feed the correct previous target word at each decoder step (not the model's own prediction) during training.

- Optimizer: **Adam** (adaptive learning rate, very common in NMT)
- Learning rate: 0.0001 – 0.001 typical
- Batch size: 32–256 sentence pairs
- Epochs: 10–50 depending on dataset size
- Early stopping based on validation BLEU or loss

Key: Parallel Data Required

NMT needs aligned sentence pairs. Quality and quantity both matter!

Attention Mechanism: The Math

Computing Attention

At each decoder step t :

$$e_{tj} = a(s_{t-1}, \mathbf{h}_j) \quad (\text{alignment score: how much does } \mathbf{h}_j \text{ match } s_{t-1}?) \quad (14)$$

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^{T_x} \exp(e_{tk})} \quad (\text{softmax normalization: attention weights}) \quad (15)$$

$$\mathbf{c}_t = \sum_{j=1}^{T_x} \alpha_{tj} \mathbf{h}_j \quad (\text{context vector: weighted sum of encoder states}) \quad (16)$$

Alignment function $a(\cdot, \cdot)$ (Bahdanau style):

$$a(s_{t-1}, \mathbf{h}_j) = \mathbf{v}_a^\top \tanh(W_a s_{t-1} + U_a \mathbf{h}_j) \quad (17)$$

$\mathbf{v}_a$, W_a , U_a are **learned parameters** (the attention model).

Attention Weights —Intuition

What attention weights α_{tj} mean:

- $\alpha_{tj} \approx 1$: decoder position t strongly attends to encoder position j
- $\alpha_{tj} \approx 0$: position j is ignored at step t
- The attention matrix can be **visualized** as a heat map
- Provides **interpretability**: we can see which source words influenced each target word

Key Benefit

Attention effectively learns **soft word alignment** simultaneously with translation — no separate GIZA++!

Myanmar-English Attention

Translating “ကျွန်တော် ကွန်ပျူတာ သုံးသည်” → “I handle the computer”

When generating “handle”, attention focuses on သုံး

When generating “computer”, attention focuses on ကွန်ပျူတာ

Full BiLSTM NMT Architecture

Complete model (Bahdanau et al., 2015):

- 1 **Embedding**: map source words to dense vectors $\mathbf{x}_t$
- 2 **BiLSTM Encoder**: compute $\mathbf{h}_j = [\vec{\mathbf{h}}_j; \overleftarrow{\mathbf{h}}_j]$ for all positions j
- 3 **Attention**: compute context vector $\mathbf{c}_t$ for each decoder step t
- 4 **LSTM Decoder**: compute state $\mathbf{s}_t = f(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}_t)$
- 5 **Output projection**: $P(y_t) = \text{softmax}(W_o \mathbf{s}_t)$

Key Results (WMT English-French)

BiLSTM with attention achieved BLEU of **28.45**, outperforming all SMT baselines at the time (2015). For the first time, a pure neural model was competitive!

Remaining Issue

Training is slow; long sequences still challenging. Solved by **Transformer (2017)**.

Beam Search in NMT Decoding

NMT Decoding

Find the most likely target sequence:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}} \sum_{t=1}^T \log P(y_t | y_1, \dots, y_{t-1}, \mathbf{x}) \quad (18)$$

Greedy decoding: always pick $\arg \max P(y_t | \dots)$ at each step. Beam search: maintain top- b partial hypotheses.

NMT Beam Search:

- Beam size $b = 5$ typical (much smaller than SMT)
- Hypotheses terminated by $\langle /s \rangle$ token
- **Length normalization**: divide score by output length $^\alpha$ (prevents short outputs)
- **Coverage penalty**: ensure all source words are attended to

```
1 # Beam search (conceptual)
2 beam = [("", 0.0)]
3 for step in range(max_len):
4     candidates = []
5     for hyp, score in beam:
6         next_probs = model.decode(hyp)
7         for w, p in top_k(next_probs):
8             candidates.append(
9                 (hyp + w, score + log(p)))
10    beam = sorted(candidates)[:b]
```

Self-Attention: The Core Idea

Self-Attention (Scaled Dot-Product Attention)

Each word in the sequence **attends to all other words** to compute its representation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (19)$$

- Q (Query): what this position is looking for
- K (Key): what each position offers
- V (Value): the actual content at each position
- d_k : dimension of key vectors (scaling prevents vanishing gradients)

Intuition: Dot product QK^T measures **similarity** (how relevant each key is to the query). Softmax converts to weights. V is the weighted sum = the output.

Example

For the word “bank” in “I went to the bank to deposit money”, self-attention helps it look at “deposit” and “money” to resolve ambiguity.

Multi-Head Attention

Run h **attention heads** in parallel, each with different learned projections:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (20)$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (21)$$

$W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$: learned projection matrices for head i

Why multiple heads?

- Different heads can attend to different **types of relationships**:
 - Head 1: syntactic dependencies (subject-verb)
 - Head 2: semantic relationships (antecedents of pronouns)
 - Head 3: positional relationships
- $h = 8$ **heads** in the original paper, $d_k = d_{\text{model}}/h = 64$

Positional Encoding

Problem: Self-attention is **permutation-invariant** — it does not know the position of each word!

Solution: Positional Encoding

Add **positional signals** to word embeddings before feeding to the Transformer:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (22)$$

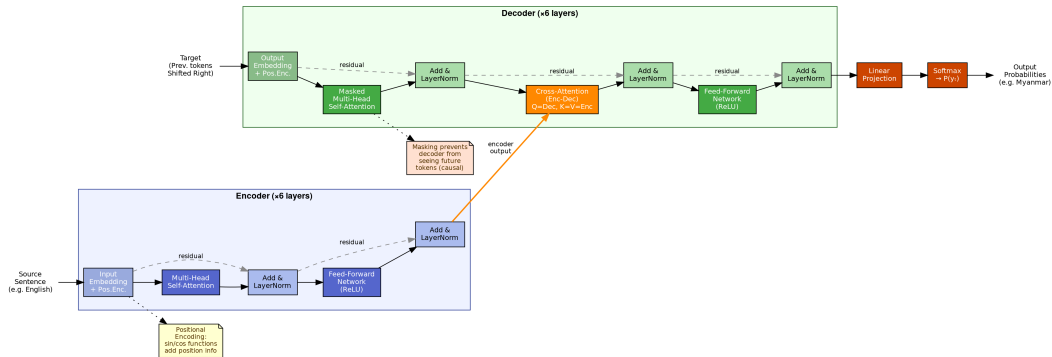
$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (23)$$

where pos is the position index and i is the dimension index.

Why sinusoidal?

- Different frequencies encode different positional information
- Model can generalize to longer sequences than seen in training
- Relative positions can be represented as linear combinations
- **Alternative:** learned positional embeddings (also common in practice)

Transformer Architecture



Transformer: Encoder Sublayers

The Transformer **encoder** consists of $N = 6$ identical layers, each with:

Encoder Layer

① **Multi-Head Self-Attention sublayer:**

$$\mathbf{h} = \text{LayerNorm}(\mathbf{x} + \text{MultiHead}(\mathbf{x}, \mathbf{x}, \mathbf{x}))$$

② **Feed-Forward sublayer:**

$$\mathbf{h}' = \text{LayerNorm}(\mathbf{h} + \text{FFN}(\mathbf{h}))$$

$$\text{FFN}(\mathbf{h}) = \max(0, \mathbf{h}W_1 + b_1)W_2 + b_2 \text{ (ReLU, inner dim=2048)}$$

Residual connections + Layer Normalization at each sublayer:

- Residual: $\text{output} = \mathbf{x} + \text{sublayer}(\mathbf{x})$ (prevents vanishing gradients in deep networks)
- LayerNorm: normalizes each sample's activations (stable training)

Transformer: Decoder

The Transformer **decoder** also has $N = 6$ layers, each with **three** sublayers:

Decoder Layer

- 1 **Masked Multi-Head Self-Attention:**
Attends to previous target positions only (causal masking prevents “peeking at the future”)
- 2 **Cross-Attention (Encoder-Decoder Attention):**
 Q from decoder, K and V from encoder output.
This is how the decoder “reads” the source sentence.
- 3 **Feed-Forward sublayer** (same as encoder)

Final output: linear layer + softmax over vocabulary $\Rightarrow P(y_t)$

Key Hyperparameters

Base model: $d_{model} = 512$, $h = 8$, $d_{ff} = 2048$, $N = 6$, 65M params

Big model: $d_{model} = 1024$, $h = 16$, $d_{ff} = 4096$, $N = 6$, 213M params

Transformer Training Details

Optimizer: Adam with custom learning rate schedule:

$$lr = d_{model}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5}) \quad (24)$$

- Warmup steps: 4000
- Learning rate **increases** linearly during warmup
- Then **decreases** proportionally to $\text{step}^{-0.5}$

Regularization:

- **Dropout:** $p = 0.1$ on all sublayers
- **Label smoothing:** $\epsilon = 0.1$ (soft targets)

Results (WMT 2014)

Model	BLEU (En-De)
Best SMT	20.9
GNMT (Google)	26.3
Transformer (base)	27.3
Transformer (big)	28.4

Trained in **12 hours** on 8 P100 GPUs!
(GNMT took weeks)

Byte Pair Encoding (BPE) for Subwords

BPE (Sennrich, Haddow, Birch, 2016)

Problem: Out-of-vocabulary (OOV) words; large vocabularies.

Solution: Represent words as sequences of **subword units**.

BPE Algorithm:

- 1 Start with character-level vocabulary
- 2 Repeatedly merge the most frequent **adjacent pair** of symbols
- 3 Stop after N merges (e.g., $N = 32,000$)

BPE Example

"Myanmar" → "Myan@@ mar" (2 subwords)

"untranslatable" → "un@@ trans@@ lat@@ able"

Key: Even unseen words can be represented! Character "@@" indicates continuation (not end of word).

SentencePiece (Kudo & Richardson, 2018): language-independent alternative to BPE (no tokenization needed); widely used for multilingual models.

Transformer for Myanmar Languages

Why Transformer works well for Myanmar:

- Myanmar has complex morphology; BPE handles this well
- Self-attention captures long-range dependencies
- Myanmar SOV $\leftrightarrow$ English SVO: cross-attention handles reordering
- Syllable-level tokenization + BPE is effective

Related work (Ye Kyaw Thu et al.):

- NMT for Myanmar–Rakhine (NAACL 2019)
- NMT for Myanmar dialects (2020)
- Thai–Myanmar–English NMT (TALLIP 2024)
- Improving NMT with POS-tag features (Heliyon 2022)

```
1 # OpenNMT-py training
2 python train.py \
3   -data data/my-en \
4   -save_model models/my-en \
5   -world_size 1 \
6   -gpu_ranks 0 \
7   -train_steps 100000 \
8   -valid_steps 5000 \
9   -encoder_type transformer \
10  -decoder_type transformer \
11  -enc_layers 6 \
12  -dec_layers 6 \
13  -heads 8 \
14  -transformer_ff 2048 \
15  -dropout 0.1
```

Back-Translation: Using Monolingual Data

Sennrich, Haddow, & Birch (2016) — ACL 2016

Back-translation (BT): use a target→source system to translate monolingual target data, then use it as additional parallel training data.

Monolingual English $\xrightarrow{\text{reverse MT (en} \rightarrow \text{my)}}$ *Synthetic Myanmar* $\Rightarrow$ Add as training pair (syn_my, real_en)

Why BT is powerful:

- Billions of monolingual sentences available
- Dramatically improves NMT, especially for low-resource
- Helps the model learn better representations of target language
- Easy to implement (just run inference backward)

Result

BT can improve BLEU by **3–10 points** for low-resource language pairs.
Essential for Myanmar ethnic language NMT where parallel data is scarce.

Marian NMT: Fast Neural MT

Marian NMT (Junczys-Dowmunt et al., 2018)

Marian is a C++ framework for neural MT, initially a reimplementation of Nematus, now supporting the full Transformer architecture. Named after **Marian Rejewski**, Polish mathematician who helped break Enigma.

Key features:

- Very **fast training and inference** (C++ backend)
- **GPU and CPU** support
- Self-contained: minimal dependencies
- Supports **Transformer, RNN, CNN** architectures
- **Multi-GPU** and multi-node training
- **Quantization** for fast CPU deployment
- Used in production at Microsoft Translator

Official Resources

- Website: <https://marian-nmt.github.io>
- GitHub:
<https://github.com/marian-nmt/marian>
- Paper: Junczys-Dowmunt et al., ACL 2018

Marian NMT: Training

Preprocess with SentencePiece:

```
spm_train --input=train.en,train.my \  
  --model_prefix=sp \  
  --vocab_size=32000 \  
  --character_coverage=0.9995 \  
  --model_type=bpe  
  
spm_encode --model=sp.model \  
  --output_format=piece \  
  < train.en > train.sp.en
```

Train Transformer:

```
marian --type transformer \  
  --train-sets train.sp.my \  
             train.sp.en \  
  --valid-sets dev.sp.my dev.sp.en \  
  --vocabs sp.vocab sp.vocab \  
  --mini-batch-fit \  
  --workspace 7000 \  
  --model model.npz \  
  --early-stopping 10
```

Decode/Translate:

```
cat test.sp.my | \  
marian-decoder \  
  --model model.npz \  
  --vocabs sp.vocab sp.vocab \  
  --beam-size 5 \  
  --normalize 1 \  
  > test.out.sp.en  
  
# Post-process  
spm_decode --model=sp.model \  
  < test.out.sp.en > test.out.en
```

Tip

Use --fp16 for half-precision training (2x faster, same quality).

From NMT to Pretrained Language Models

The New Paradigm: Pretrain then Fine-tune

Rather than training from scratch for each language pair, modern MT uses:

- 1 **Pretraining:** train a large model on massive multilingual data to learn general language representations
- 2 **Fine-tuning:** adapt the pretrained model to a specific translation direction using parallel data

Key advantages:

- **Transfer learning:** knowledge from high-resource languages transfers to low-resource languages
- **Less parallel data** needed for fine-tuning
- **Better representations:** pretrained on orders of magnitude more text than any MT training set
- **Zero-shot** and **few-shot** translation possible

XLM-R: Cross-Lingual Language Model

XLM-R (Conneau et al., 2020)

XLM-RoBERTa: trained on 100 languages using **Masked Language Modeling only** (better than NSP), on **2.5 TB of CommonCrawl data**.

- Significantly outperforms mBERT on all cross-lingual benchmarks
- Available in Base (270M) and Large (560M) sizes
- Key improvement: much more data + better multilingual balance
- Good for fine-tuning on low-resource language pairs
- Supports **Myanmar** (included in 100 languages)

Fine-tuning XLM-R for MT (Conceptual)

```
1 from transformers import XLMRobertaModel, XLMRobertaTokenizer
2 tokenizer = XLMRobertaTokenizer.from_pretrained("xlm-roberta-base")
3 model = XLMRobertaModel.from_pretrained("xlm-roberta-base")
4 # Add a cross-attention decoder on top
5 # Fine-tune on parallel corpus
```

mBART: Multilingual Denoising Pretraining

mBART (Liu et al., 2020)

Multilingual BART: sequence-to-sequence model pretrained with **denoising autoencoding** on **25 languages**.

- Based on BART architecture: Transformer encoder + decoder
- Pretraining: corrupt input (masking, deletion, permutation) → reconstruct original
- Supports both **encoder and decoder** ⇒ ideal for MT
- mBART-50: extended to 50 languages (including Myanmar)
- Fine-tune for any language pair (even zero-shot between seen languages)

Why mBART is good for MT

Unlike BERT (encoder only), mBART has a full encoder-decoder, so fine-tuning for MT is natural. Just add source and target language tokens: [MY_LANG] source sentence [EN_LANG] → target sentence

T5: Text-to-Text Transfer Transformer

T5 (Raffel et al., 2020)

T5 (Text-to-Text Transfer Transformer, Google): frames **all NLP tasks** as text-to-text problems.

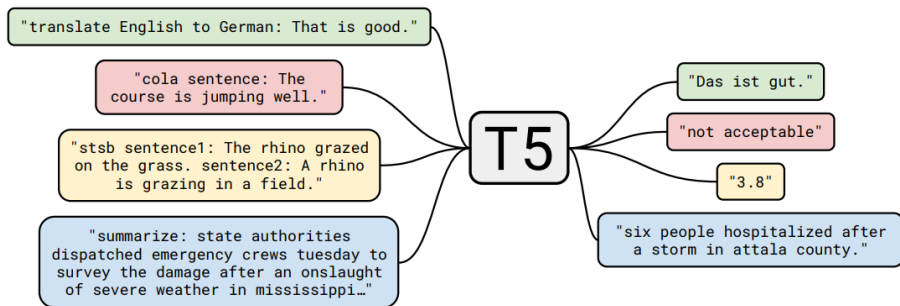
- Trained on **750 GB of clean web text** (C4 dataset)
- For translation: input = “translate English to Myanmar: {sentence}”
- Output = Myanmar translation
- Available in Small (60M) to 11B parameter versions

mT5 (Xue et al., 2021)

Multilingual T5: T5 trained on **101 languages** (mC4 dataset).

- 300M – 13B parameters
- Includes Myanmar language
- Fine-tune with: “translate my to en: မြန်မာ → Myanmar”

T5: Text-to-Text Transfer Transformer



Text-to-Text Transfer Transformer

Overview of the text-to-text paradigm. By framing different NLP tasks as text-to-text problems, the model simplifies the development pipeline. The same architecture, loss function, and hyperparameters are applied across all tasks. (Source: Raffel et al., 2019)

T5 Unified Framework

MT, summarization, QA, sentiment all use the same model format. Great for **multi-task learning** (one model for many tasks).

M2M-100: Many-to-Many MT

M2M-100 (Fan et al., 2020, Meta AI)

M2M-100: first MT model trained for **any-to-any** translation across **100 languages** (10,000 language pairs).

- 1.2B parameters (large model)
- Does not require English as a pivot language
- Training data: 7.5B sentence pairs (web-mined + curated)
- Uses language tokens: `__my__` for Myanmar

Why non-English pivoting matters:

- Many languages are related but one or both are not English (e.g., Myanmar↔Thai)
- Direct translation is more accurate than pivot through English
- Fewer error propagations

Usage

```
1 from transformers import
   M2M100ForConditionalGeneration, \
2   M2M100Tokenizer
3 model = M2M100ForConditionalGeneration\
4   .from_pretrained("facebook/m2m100_418M")
5 tokenizer = M2M100Tokenizer\
6   .from_pretrained("facebook/m2m100_418M")
7 tokenizer.src_lang = "my"
```

NLLB: No Language Left Behind —Introduction

NLLB (Costa-jussà et al., 2022, Meta AI)

NLLB-200: Meta AI's landmark multilingual MT model supporting **200 languages**, including 149 previously underserved languages.

"No Language Left Behind" — democratizing MT for the world.

- Published: July 2022 (arXiv: 2207.04672)
- Models available: 600M, 1.3B, 3.3B parameters
- Covers **Myanmar** and several Myanmar ethnic languages
- Open-sourced weights on HuggingFace
- Part of Meta AI's effort for AI equity across languages

Impact

NLLB demonstrated that even the world's most under-resourced languages (some with <1M speakers) can benefit from neural MT.

Technical Innovations:

- ① **LASER3**: extended sentence encoder for 200+ languages (used for mining and filtering data)
- ② **FLORES-200**: new benchmark for 200 languages (1012 sentence evaluation set)
- ③ **Sparse MoE**: Mixture of Experts routing for 54B model scales parameters without proportional compute cost
- ④ **Toxicity filtering**: remove harmful content from training data

NLLB Results

Language	NLLB	Prev best
Burmese	+50%	—
Shan	+70%	—
Rakhine	New!	—
Kachin	New!	—

(Relative BLEU improvements on FLORES-200)

NLLB: Myanmar and Ethnic Languages

Languages Supported (Myanmar region)

- **Burmese (mya)**: High-resource in NLLB
- **Shan (shn)**: Major ethnic language included
- **Rakhine (rki)**: Arakanese dialect included
- **Sgaw Karen (ksw)**: Karen language included
- **Mon (mnw)**: Mon-Khmer language included
- **Lahu (lhu), Chin (cnh)**: also included

How NLLB helps Myanmar NLP research:

- Pre-trained weights can be **fine-tuned** on domain-specific data
- Use as strong **baseline** for comparison
- Mine parallel data with LASER3
- Transfer learning for even smaller Myanmar ethnic languages

```
1 from transformers import pipeline
2
3 translator = pipeline(
4     "translation",
5     model="facebook/nllb-200-distilled-600M"
6 )
7 result = translator(
8     "I love Myanmar", src_lang="eng_Latn",
9     tgt_lang="mya_Mymr"
10 )
11 print(result[0]['translation_text'])
```

NLLB: Fine-tuning for Domain Adaptation

```
1 from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
2 from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments
3
4 # Load pre-trained NLLB model
5 model_name = "facebook/nllb-200-distilled-600M"
6 model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
7 tokenizer = AutoTokenizer.from_pretrained(model_name)
8
9 # Fine-tune on Myanmar medical domain data
10 training_args = Seq2SeqTrainingArguments(
11     output_dir="./nllb-my-medical",
12     per_device_train_batch_size=16,
13     num_train_epochs=3,
14     learning_rate=5e-5,
15     warmup_steps=500,
16     weight_decay=0.01,
17     predict_with_generate=True,
18     fp16=True, # Mixed precision for speed
19 )
20
21 trainer = Seq2SeqTrainer(
22     model=model,
23     args=training_args,
24     train_dataset=train_dataset,
25     eval_dataset=eval_dataset,
26 )
27 trainer.train()
```

Comparison: MT Systems 2024

System	Type	Languages	Quality	Cost
Moses	SMT	Any (with data)	Moderate	Low (CPU)
Marian	NMT	Any	Good	Medium (GPU)
OpenNMT	NMT	Any	Good	Medium
mBART-50	Pretrained	50	Very good	Medium
M2M-100	Pretrained	100	Very good	Medium
NLLB-200	Pretrained	200	Excellent	Medium
DeepL	Commercial	33	Excellent	API fee
Google Translate	Commercial	133	Excellent	API fee
ChatGPT/GPT-4	LLM	100+	Near-human	High
Gemini	LLM	74+	Near-human	High

For Myanmar NLP Research

NLLB-200 + fine-tuning is currently the best starting point. Use Moses as a baseline. Use ChatGPT for data augmentation (careful!).

Reinforcement Learning for MT Fine-tuning

RLHF for MT (Recent Trend)

Reinforcement Learning from Human Feedback (RLHF) can further improve MT quality by training models to produce translations preferred by humans.

- Step 1: Collect **human preferences** between MT outputs
- Step 2: Train a **reward model** to predict human preferences
- Step 3: Fine-tune the MT model with **PPO** to maximize reward

Research: RL for Myanmar Dialect MT

Ye Kyaw Thu, Thazin Myint Oo, Thepchai Supnithi (2023):

“Reinforcement Learning Fine-tuning for Improved Neural Machine Translation of Burmese Dialects” (Workshop M3Oriental, EMNLP 2023)

Applied RL fine-tuning to improve NMT between Burmese and Rakhine dialect with small parallel corpus. RL-based fine-tuning improved fluency and adequacy.

Myanmar Sign Language MT:

- **Swe Zin Moe et al. (2020):**
Unsupervised NMT between Myanmar Sign Language and Myanmar text
- **NMT for MSL:** using sequence-to-sequence models
- Challenge: sign language has its own grammar, visual modality

Myanmar Braille MT:

- **Zun Hlaing Moe et al. (2021):**
Myanmar text to Braille (Mu Thit) translation using IBM Model 1 and 2
- Braille codes mapped to IBM translation model

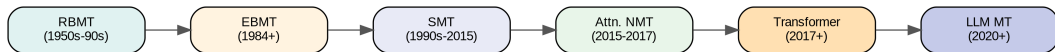
Interesting Challenges

- Sign languages are **spatial** and **visual**
- No standard written form for most sign languages
- Need specialized corpora (video, gloss)
- Myanmar Sign Language is different from ASL/BSL

Khmer Braille (2024)

Kimhuoy Yann, Ye Kyaw Thu et al. (CITA 2024): SMT vs. NMT for Khmer Braille translation.

Journey Through MT: Summary



Key lessons:

- ① Rules → Statistics → Neural: data-driven beats hand-crafted
- ② Better architectures: word → phrase → seq2seq → Transformer
- ③ Scaling works: more data and parameters consistently improve quality
- ④ Low-resource remains a challenge; transfer learning is the key
- ⑤ Evaluation is hard; human judgment is the gold standard

Selected Contributions to MT (Ye Kyaw Thu et al.)

SMT Research

- Phrase-based, Hierarchical, OSM for Myanmar–Pa'O (*JlIST 2021*)
- SMT for Myanmar–Rakhine, Burmese–Dawei/Tavoyan, Myanmar–Myeik (*NSURL 2019, ICCA 2020*)
- SMT for Myanmar ethnic languages: Kachin-Rawang, Kayah, Chin, Shan (*iSAI-NLP 2019, iSAI-NLP 2020, COCOSA 2020, JlIST 2020*)
- IBM Model 1&2 for Myanmar Text (Burmese) and Braille (*JlIST 2021*)
- SMT between Myanmar Sign Language and Myanmar text (*ICCA 2018*)
- SMT between Myanmar Sign Language and Myanmar SignWriting (*ASEAN-AI 2018*)
- Large-scale SMT study for Myanmar language (*SNLP 2016*)
- A Study of SMT Methods for Under Resourced Languages (*SLTU 2016*)

NMT Research

- NMT between Myanmar Sign Language and Myanmar Written Text (*ONA 2018*)
- NMT for Myanmar–Rakhine (Arakanese) (*VarDial@NAACL 2019*)
- NMT for Myanmar dialects (Dawei) (*ICCA 2020, JIIST 2020*)
- Unsupervised NMT for Myanmar Sign Language (*JIIST 2020*)
- Improving NMT with POS-tag features for low-resource pairs (*Heliyon 2022*)
- Thai–Myanmar–English multilingual NMT (*TALLIP 2024*)
- RL fine-tuning for Burmese dialect NMT (*M3Oriental@EMNLP 2023*)
- Hybrid SMT for English–Myanmar at WAT-2021 (*WAT 2021*)

COMET: Neural MT Evaluation

COMET (Rei et al., 2020)

Crosslingual Optimized Metric for Evaluation of Translation: uses a **neural model** fine-tuned on human judgments to score translations.

- Based on XLM-RoBERTa (multilingual pretrained encoder)
- Input: source sentence + MT output + reference translation
- Output: quality score (0–1, higher = better)
- Much higher correlation with human judgments than BLEU
- **COMET-22** and **BLEURT** are state-of-the-art automatic metrics

Metric Correlation with Humans

Metric	Pearson r (WMT21)
BLEU	0.63
TER	0.55
chrF	0.67
METEOR	0.64
COMET-22	0.87

Usage

```
pip install unbabel-comet
comet-score \
  -s source.txt \
  -t hypothesis.txt \
  -r reference.txt
```

FLORES-200: The Standard Benchmark

FLORES (Few-shot Learning Across Languages)

FLORES-200: evaluation benchmark for 200 languages created by Meta AI.

- **1012 sentences** per language (from WikiNews)
- Split: dev (997 sentences) + devtest (1012 sentences)
- Professionally translated (not machine-generated)
- Covers all 200 NLLB languages
- Includes **Myanmar (mya)**, Shan (shn), Rakhine (rki), Karen (ksw), Mon (mnw)

How to use FLORES-200:

- 1 Download from GitHub
- 2 Translate `devtest.eng_Latn` with your system
- 3 Compare output against `devtest.mya_Mymr`
- 4 Report sacrebleu BLEU, chrF, COMET

```
pip install sacrebleu
# Compute BLEU
sacrebleu devtest.mya_Mymr \
  -i output.mya \
  -m bleu chrF

# chrF++ (recommended for Myanmar)
sacrebleu devtest.mya_Mymr \
  -i output.mya \
  -m chrF --chrF-word-order 2
```

Word Alignment Symmetrization

Symmetrization (Och & Ney, 2003)

GIZA++ gives **directional** alignments: $e \rightarrow f$ and $f \rightarrow e$. Combining them gives better coverage:

Common symmetrization heuristics:

Intersection Keep only alignments in **both** directions (high precision, low recall)

Union Keep all alignments from either direction (high recall, low precision)

grow-diag-final-and Start with intersection; iteratively add diagonal, then non-diagonal points from union that improve coverage. Default in Moses.

Moses Command

```
# After GIZA++ runs both directions:
atools -c grow-diag-final-and \
-i fwd.A3.final \
-j rev.A3.final \
> sym.align
```

Or automatically via train-model.perl:

```
-alignment grow-diag-final-and
```

Language Understanding Lab, Myanmar:

- Pioneered parallel corpus creation for Myanmar ethnic languages
- Developed SMT and NMT systems for previously untranslated language pairs
- Word-to-word translation between Myanmar and ethnic languages
(*May Myat Myat Khaing et al., JIIST 2022*)
- English to Burmese MT with Asian pivot languages
(*Wint Theingi Zaw et al., JIIST 2022*)
- Myanmar Sign Language translation
(*Hlaing Myat Nwe et al., MT Summit 2023*)

Datasets Released

Most of the datasets available at:
<https://github.com/ye-kyaw-thu>

- myPar (ethnic) (ပြင်ဆင်နေဆဲ)
- Asian Language Treebank (ALT) Myanmar
- MSL4Emergency sign language corpus
- myBraille corpus (ပြင်ဆင်နေဆဲ)
- Burmese dialect corpus

Day 2: Hands-on SMT and NMT

We will work with parallel corpora developed by Dr. Ye Kyaw Thu.

Practical SMT (Moses):

- 1 Prepare Myanmar–English parallel corpus
- 2 Train language model (KenLM)
- 3 Run word alignment (GIZA++)
- 4 Extract phrase table
- 5 Tune weights (MERT)
- 6 Evaluate with BLEU

Practical NMT (OpenNMT-py / Marian):

- 1 Preprocess with SentencePiece
- 2 Configure Transformer model
- 3 Train on GPU
- 4 Decode test set
- 5 Evaluate with BLEU, chrF
- 6 Compare SMT vs. NMT results

Datasets to Use

Myanmar–Thai corpus from Core AI Resarch Team, NECTEC, Thailand.

Research Directions:

- ① **Very low-resource MT:** 100–1000 sentence pairs
- ② **Document-level MT:** context beyond sentences
- ③ **Multimodal MT:** text + images (visual MT)
- ④ **Speech-to-speech** translation (direct, no text)
- ⑤ **Real-time / simultaneous** MT (before utterance ends)
- ⑥ **Interpretable NMT:** understanding attention and decisions
- ⑦ **Sustainable MT:** reduce carbon footprint of training

Myanmar-Specific Challenges:

- ① MT for endangered ethnic languages (Rawang, Palaung, Lahu)
- ② Sign Language MT (Myanmar Sign Language)
- ③ Braille MT for accessibility
- ④ Dialect translation (Rakhine, Dawei, Mandalay)
- ⑤ Code-switching (Myanmar + English)
- ⑥ Myanmar→Thai for border communities
- ⑦ LLM evaluation for Myanmar language quality

Key References: SMT

- Brown, P.F., et al. (1993). *The mathematics of statistical machine translation*. Computational Linguistics, 19(2), 263–311.
- Koehn, P., Och, F.J., & Marcu, D. (2003). *Statistical phrase-based translation*. HLT-NAACL 2003.
- Och, F.J. (2003). *Minimum error rate training in statistical machine translation*. ACL 2003.
- Chiang, D. (2007). *Hierarchical phrase-based translation*. Computational Linguistics, 33(2).
- Durrani, N., et al. (2011). *A joint sequence translation model with integrated reordering*. ACL 2011.
- Koehn, P., et al. (2007). *Moses: Open source toolkit for SMT*. ACL 2007 (demo).
- Papineni, K., et al. (2002). *BLEU: a method for automatic evaluation of MT*. ACL 2002.
- Heafield, K. (2011). *KenLM: Faster and smaller language model queries*. WMT 2011.
- Koehn, P. (2009). *Statistical Machine Translation*. Cambridge University Press. (Textbook)

Key References: NMT and Pretrained Models

- Cho, K., et al. (2014). *Learning phrase representations using RNN encoder-decoder for SMT*. EMNLP 2014.
- Sutskever, I., Vinyals, O., & Le, Q.V. (2014). *Sequence to Sequence Learning with NNs*. NIPS 2014.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural MT by jointly learning to align and translate*. ICLR 2015.
- Vaswani, A., et al. (2017). *Attention is all you need*. NIPS 2017.
- Sennrich, R., et al. (2016). *Neural MT of rare words with subword units*. ACL 2016.
- Liu, Y., et al. (2020). *Multilingual denoising pre-training for NMT*. TACL 2020.
- Costa-jussà, M.R., et al. (2022). *No Language Left Behind*. arXiv:2207.04672.
- Fan, A., et al. (2021). *Beyond English-centric multilingual MT*. JMLR 2021.
- Junczys-Dowmunt, M., et al. (2018). *Marian: Fast NMT*. ACL 2018.
- Koehn, P. (2017). *Neural Machine Translation*. arXiv:1709.07809.

Resources and Useful Links

Software and Toolkits:

- Moses: <http://statmt.org/moses>
- KenLM:
<https://kheafield.com/code/kenlm>
- fast_align:
https://github.com/clab/fast_align
- OpenNMT: <http://opennmt.net>
- Marian:
<https://marian-nmt.github.io>
- Fairseq: <https://github.com/facebookresearch/fairseq>
- HuggingFace: <https://huggingface.co>

Data and Benchmarks:

- OPUS: <http://opus.nlpl.eu>
- FLORES-200: <https://github.com/facebookresearch/flores>
- WMT: <http://statmt.org/wmt>
- MT-Bench: <https://huggingface.co/spaces/lmsys/mt-bench>

Myanmar NLP Resources of LU Lab., Myanmar:

- <https://github.com/ye-kyaw-thu>
- <https://sites.google.com/site/yekyawthunlp>

Thank You!

Questions? Comments?
ykt.nlp.ai@gmail.com

Appendix

Extra Notes & Additional Information

Why MT Evaluation is Hard

- There is **no single correct translation** —many valid translations exist
- Human evaluation is the gold standard but **expensive and slow**
- Automatic metrics enable fast feedback during development
- Key criteria for a good MT output:
 - ① **Adequacy**: Does the output convey the same meaning as the source?
 - ② **Fluency**: Is the output grammatically correct and natural?
 - ③ **Fidelity**: Are named entities, numbers, and facts preserved?

Important Caveat

No automatic metric perfectly correlates with human judgment. Always report multiple metrics and back them up with human evaluation for important decisions or research publication.

BLEU Score: Details and Calculation

Modified N-gram Precision

For each n-gram order n , compute the **modified precision** p_n :

$$p_n = \frac{\sum_{\bar{e} \in \text{output}} \min(\text{count}(\bar{e}, \text{output}), \max_{\text{ref}} \text{count}(\bar{e}, \text{ref}))}{\sum_{\bar{e} \in \text{output}} \text{count}(\bar{e}, \text{output})} \quad (25)$$

“Clipping”: each n-gram counted at most as many times as it appears in any reference.

BLEU Step-by-Step

Reference: “I will go to Yangon tomorrow”, **MT output:** “I will go Yangon tomorrow”

n-gram	Output matches	Clipped count	Total output n-grams	p_n
1	I, will, go, Yangon, tomorrow	5	5	1.00
2	“I will”, “will go”, “Yangon tomorrow”	3	4	0.75
3	“I will go”	1	3	0.33
4	none	0	2	0.00

$\text{BLEU} = \text{BP} \cdot \exp(0.25 \cdot (\log 1 + \log 0.75 + \log 0.33 + \log 0)) \approx 0$

(Zero because $p_4 = 0$ — the log penalty dominates)